

Slash commands and model selection

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN Session 5 of 12 · English notes

The same recap that covered permission modes covered **slash commands**: built-in Claude Code commands (`/model`, `compact`, and the rest of the in-product list) versus **custom skills** that appear as slash entries once you add a `SKILL.md`. Model selection is not a separate product — it is a slash (or a “switch model” control) that binds the current session, and later each sub-agent, to a Haiku / Sonnet / Opus-class model your org can actually use.

COMMAND PALETTE DRIFT

The live UI showed a default around “Sonnet 5” and a `/model` picker of whatever the org had. Built-in names (`/compact`, `/clear`, `agents`, `mcp`, `hooks`) move between releases. Read the in-session `/` menu and [Claude Code docs](#) rather than memorising a 12 August screenshot.

/model — set or list, do not guess SKUs

Type `/model` (the trainer also said `/models`) to list models available to the workspace, or `/model` plus a name to switch. There is often a switch-model control showing the current default. Teaching points:

- Availability is organisational: you cannot select a model the workspace is not entitled to.
- Use a stronger model (Opus-class in the lecture) for planning / architecture-style sub-agents; Sonnet-class for routine generation; Haiku-class when the task is cheap classification or boilerplate — always re-check the live ladder.
- Sub-agents can *inherit* the parent model or pin their own in the agent markdown (M05-C03). A Docker-file agent does not need the same model as a principal-architect skill.
- Switching model does not flush `CLAUDE.md`; it does change cost and latency of the next turns.

Built-in slashes versus skill slashes

Kind	Examples from class	Who defines it	When it runs
Built-in Claude Code	<code>/model</code> , <code>compact</code> (and other session utilities the <code>/</code> menu lists)	The product	Whenever you type them; they configure the client.
Skill slash	<code>/business-analyst</code> , <code>/architect</code> , <code>/backend</code> , <code>/frontend</code> , later <code>/docker-style</code> skills	Your repo’s skill folders (<code>SKILL.md</code>)	On explicit slash, or auto if model invocation is not disabled.
Agent mention / delegate	Review the code → code-reviewer agent; generate Dockerfile → docker agent	<code>`.claude/agents`</code> (or user/plugin scope)	Orchestrator matches description, or you invoke the agent.

Skills are “custom slash commands” in the trainer’s vocabulary. They are not MCP. They inject a role packet for one turn (or until the skill’s instructions are in context). Compact is the opposite kind of command: it

shrinks session context so you can keep working without starting from a blank chat — related to context-window management from Module 03, not to picking Opus.

Configuration surfaces the / menu sits on

Slash commands are how you *drive* Claude Code; configuration still lives in files: project `CLAUDE.md`, nested `CLAUDE.md`, `settings.json` (hooks in Module 06), skill trees, agent markdown. The trainer also pointed at Claude Code configuration generally: path-specific memory, permission mode, model. Treat the slash bar as the interactive API of those files, not as a second source of truth.

Asking Claude to write the slash for you

Later in the same session the trainer asked Claude to *create a skill* (e.g. Docker/container work) rather than hand-writing every `SKILL.md`. That is allowed: the generated skill then shows up as a slash like the handmade ones. Constrain the prompt (“only the frontend container, do not start Docker Desktop and boil the ocean”) or the new skill will inherit the same over-eager agent loop you were trying to package. Disable model invocation on a skill if you need it never to fire unless slashed.

Prompt context is not only the slash

When you slash frontend and paste a screenshot of filters, the model still loads frontend `CLAUDE.md`. Better prompts name the route or component (`/tasks`) so search tokens are not spent grepping the whole SPA. README start/stop commands can be a slash-time instruction: “refer to README and start the app” instead of dumping every npm script into the skill.

EXAM-READY ONE-LINER

`/model` chooses the brain; skill slashes choose the role; permission mode chooses how often the brain must ask before using tools. Mixing those three up is a common written-answer failure.

What not to put in a slash

Do not paste client data. Do not rely on a slash to enforce Prettier or company format — that is a local hook (Module 06) so it does not bill tokens. Do not create a skill that is actually an MCP server; if you need Figma or an internal API, that is tools/MCP, optionally *allowed* on an agent, not a `SKILL.md` pretending to log in.

Official documentation

- **Claude Code overview**

<https://code.claude.com/docs/en/overview>

Slash menu, skills, and session commands including model.

- **Models**

<https://docs.claude.com/en/docs/about-claude/models>

Which IDs `/model` may list this week.

- **Workspaces**

<https://docs.claude.com/en/docs/build-with-claude/workspaces>

Why two UST users can see different model lists.

- **Prompting overview**

<https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>

Skills are packaged prompts; compact is context hygiene.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.